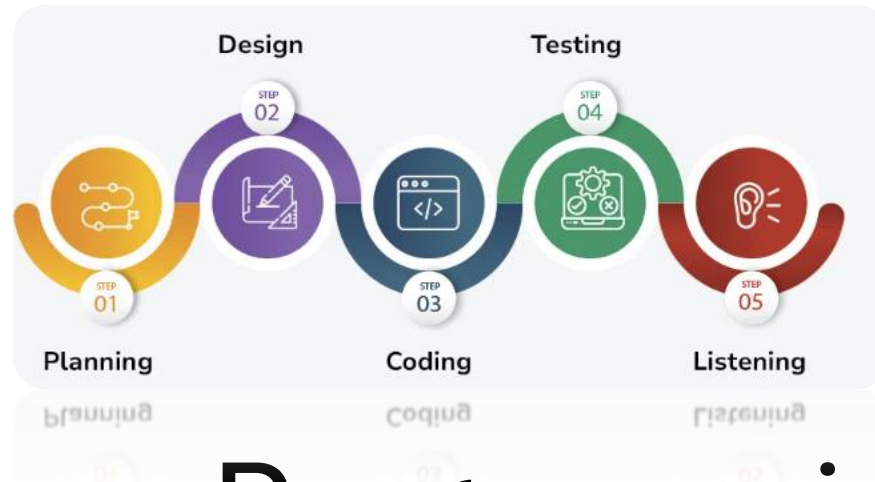




eXtreme Programming (**XP**) and Unit Testing with **JUnit**



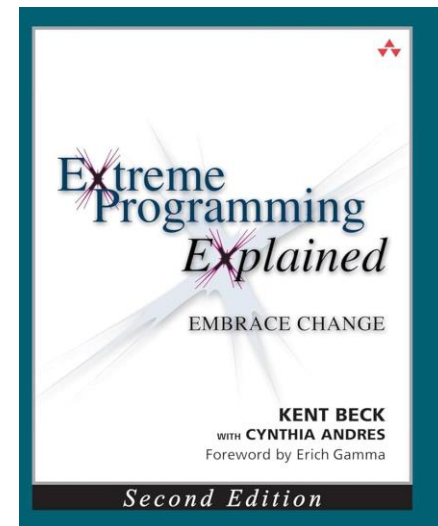
What is Agile Development?

- Family of development processes
 - **Allow the team** to respond to changes in any phase of development
 - **Accepts** change as a fact of life and uses process to enable change
- Agile Development Process: **Extreme Programming (XP)**



eXtreme Programming (XP)

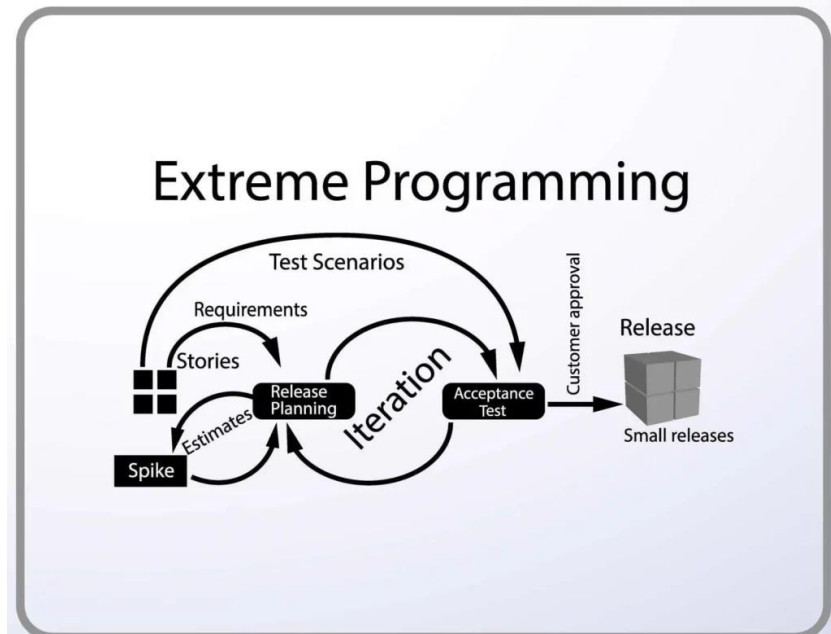
“Extreme Programming (XP) is a software development methodology that emphasizes **customer satisfaction, flexibility**, and the **delivery of high-quality software through frequent releases** and **iterative feedback**. ...



Beck, Kent. *Extreme Programming Explained: Embrace Change*. 2nd ed., Addison-Wesley, 2004.

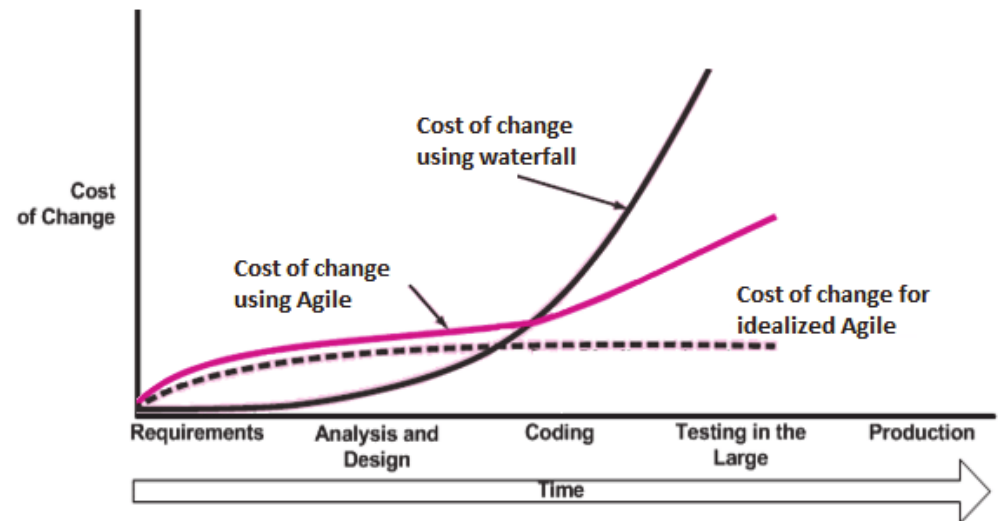
eXtreme Programming (XP)

... It works by bringing the whole team together in the presence of **simple practices**, with **enough feedback** to enable the team to see where they are and to tune the practices to their unique situation.” – Ron Jeffries

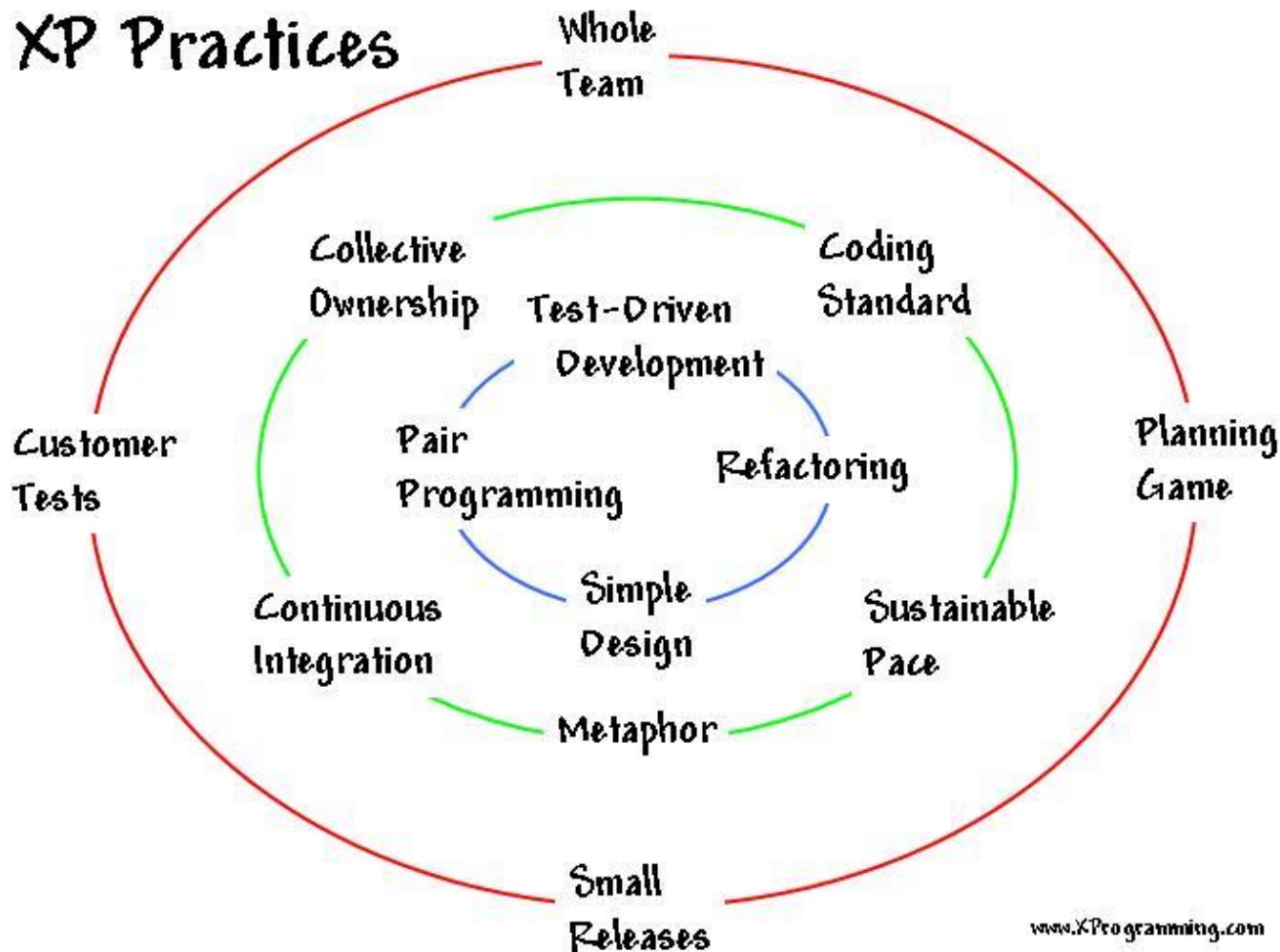


eXtreme Programming (XP)

- **Defined by 12 practices**
- Claim: XP ‘flattens’ the **cost-of-change curve**
- Most literature on XP is experience reports
 - Rigorous evaluation is needed



12 Practices of XP



XP Practices: Whole Team

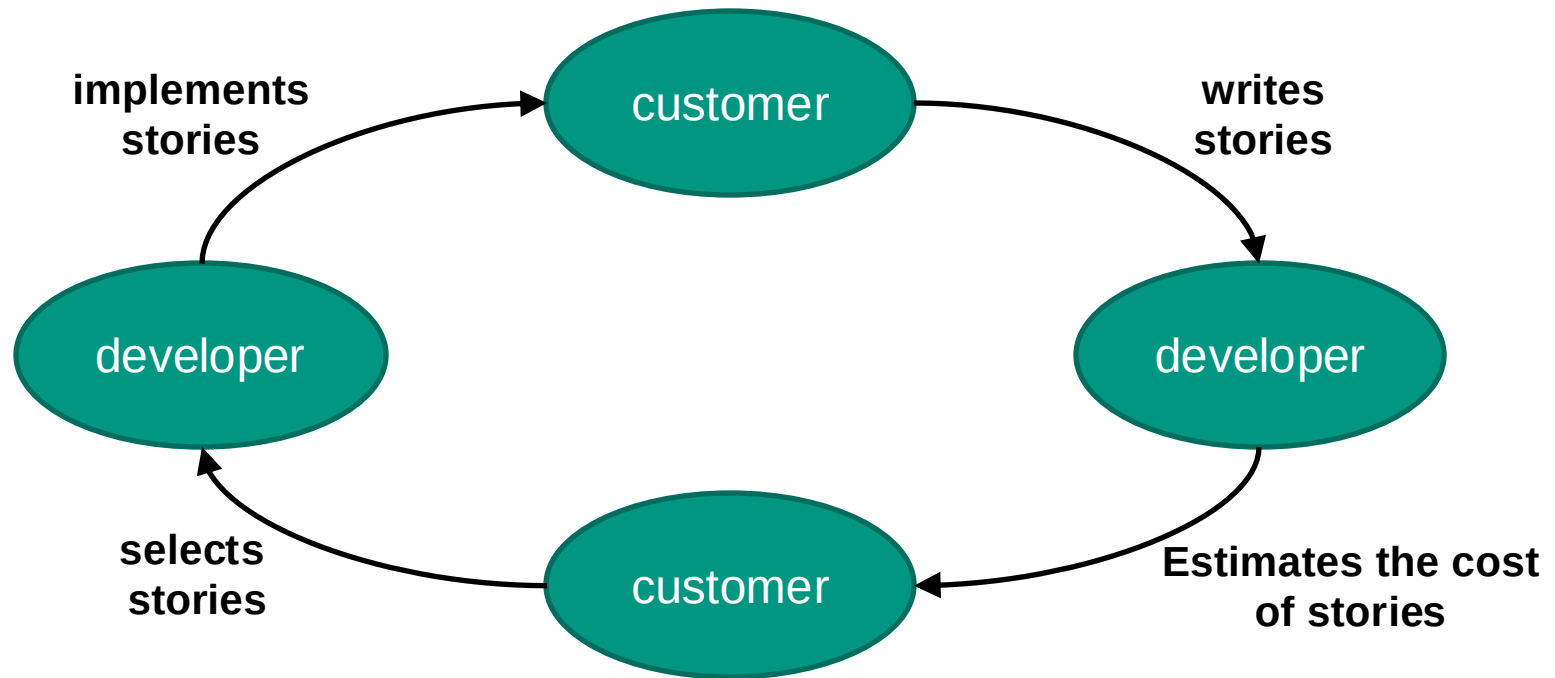
- Everyone working on project is on one team
- “Customer” – **provides requirements, steers planning**
- Different roles on team, but:
 - No specialists
 - Generally competent people with special skills

Whole Team – Customer on-site

- All contributors to an XP project are one team
- Must include a business representative – the **‘customer on-site’**
 - Provides requirements
 - Sets priorities
 - Steers project
- Team members are **programmers, testers, analysts**, coach, manager
 - Coach: supports the team in adhering to practices
 - manager: tracks the estimated and actual development effort
- Best XP teams have no specialists

Our project team members
are **programmers, testers,**
analysts!

Interaction between customer and developer



XP Practices: Planning Game

- Address two questions:
 - What will be accomplished by the due date?
 - What to do next?
- **Release Planning**
- **Iteration Planning**

Planning game

- No more freezing requirement
 - No more requirement document
- No more exact prediction
 - Predict what will be accomplished by the due date
 - Determine what to do next
- **Story cards** (user stories)
 - Customer presents required features ...
 - Developers estimate difficulty ...
 - Revise regularly

Planning game (2)

- XP Release Planning
 - Customer describes required features
 - Programmers estimate difficulty
 - Imprecise but revised regularly
- XP Iteration Planning
 - **Two week** iterations
 - Customer presents features required
 - Programmers break features down into tasks
 - Team members sign up for tasks
 - Running software at end of each iteration

Story

- A specific **system behavior** from the user's **perspective**
- No **exact specification**
- Basis for discussion
- **Implementable in an iteration**

How stories are produced?

- Customer tells **something about how to use the system**
- Developers **ask comprehension questions**
- Customer **writes story in own words**
- Over time, stories are discarded, rewritten, shared and merged
- (for our group project instructor/DIA or a team member can serve as a customer)

Story card – Example (1)

- ID and Task Description
- Priority
- Estimation
- Confirmation

Front of Card

173

As a student I want to purchase a parking pass so that I can drive to school

Priority: ~~High~~ Should
Estimate: 4

Back of Card

Confirmations:

~~The student must pay the correct amount~~
One pass for one month is issued at a time
The student will not receive a pass if the payment isn't sufficient
The person buying the pass must be a currently enrolled student.
The student may only buy one pass per month.

Story card – Example (2)

- **Examples of user stories for a parking pass system:**
- Students can purchase monthly parking passes online.
- Parking passes can be paid via credit cards.
- Parking passes can be paid via PayPal.

Story card – Example (3)

• **Example of user stories for a seminar management system:**

- Professors can input student grades.
- Students can obtain their current seminar schedule.
- Students can order official transcripts.
- Students can only enroll in seminars for which they have prerequisites.
- Transcripts will be available online via a standard browser.

XP Practices: Customer Tests

- **Automated** acceptance tests
- **Defined** by customer
- **Implemented** by team

XP Practices: Small Releases

- Release **functional**, “**useful**” software **every iteration**
 - For evaluation by customer, or release to end-users
- **Releases** are kept **reliable by testing**

XP Practices:

Continuous Integration

- **Constantly** keep the **entire system integrated**
- Multiple **daily builds** (10-20 in practice!)
- Problems with **infrequent integration**
 - Team not **experienced with integration**
 - **Buggy code** (problems introduced by integration)
 - **Long code freezes**

XP Practices:

Collective Code Ownership

- **Anyone on team can work to improve any piece of code**
- Avoids asking **code “owner”** to add feature
- Problem: **working with unfamiliar code**
 - **Pair with someone familiar with it**
 - **Automated tests**

XP Practices:

Coding Standard

- **Common coding standard** followed by everyone on team
- **Specifics unimportant** – as long as code all looks familiar
- Supports **collective code ownership**

XP Practices: Metaphor

- **Metaphor for function of system** shared by team
- The metaphor is a **shared, simple story or analogy** that helps the development team and stakeholders understand how the system works.
- For example, **an agent-based information retrieval system**:
“This program works like a hive of bees, going out for pollen and bringing it back to the hive.”
[www.xprogramming.com]

XP Practices:

Sustainable Pace

- Maintain pace that will be successful in the long run
- Pace should be **sustainable indefinitely**
- **Work overtime** when necessary, but don't **burn out and lose productivity**

XP Practices:

Pair Programming

- All code written by **two programmers working side-by-side**
- Ensures **code is reviewed**
- Communicates **knowledge throughout team**
- Claim: **results in better code**

XP Practices: Simple Design

- **Start** with a **simple design** and keep it that way through **design improvement**
- Don't make **code unnecessarily general**
- **No wasted effort** – design suited for current functionality
- Example: FSA (Financial Services Application) system, you might start by designing a simple class structure to manage account balances and transactions...

XP Practices:

Design Improvement

- Continuous process of **design improvement** called “**Refactoring**”
- Implementation of code changed without altering interface
- **Remove duplication**

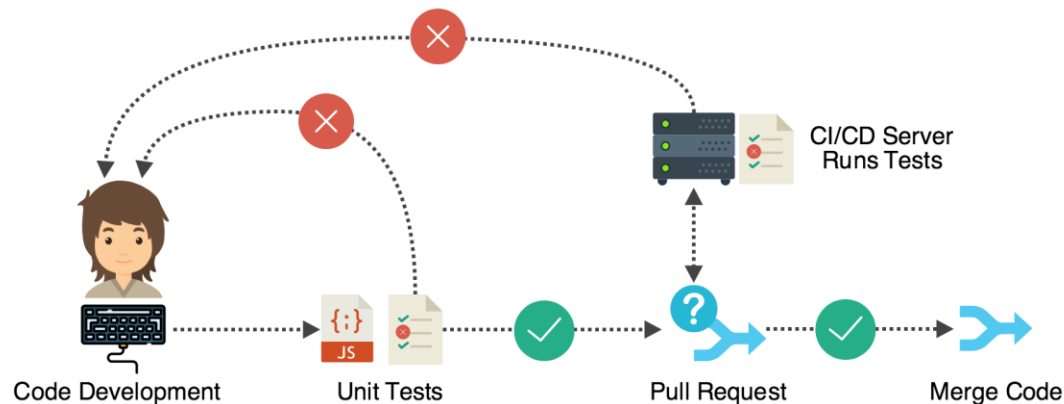
XP Practices:

Test Driven Development

- Write **test first**, then **write code to make it work**
 - A form of **Design by Contract**
- **Every test must pass at every build**
 - Supports Continuous Integration

Unit Testing

- Based on the idea that **classes should contain their own tests**
- **Highly localized**; test(s) work within a single package
- **Tests the interfaces to other packages**, but just assumes other packages work



Why Unit Testing?

- Better **able to exercise all code**
- Can **write tests before writing code**:
 - Helps programmer to focus on the interface rather than the implementation
 - Provides **a clear finish point: when the test works**
- Cuts down **significantly on debugging time**
 - Run tests every time code is compiled
 - If new code breaks a previously-passed test, **bug location is easier to pinpoint**

Unit Testing Difficulties

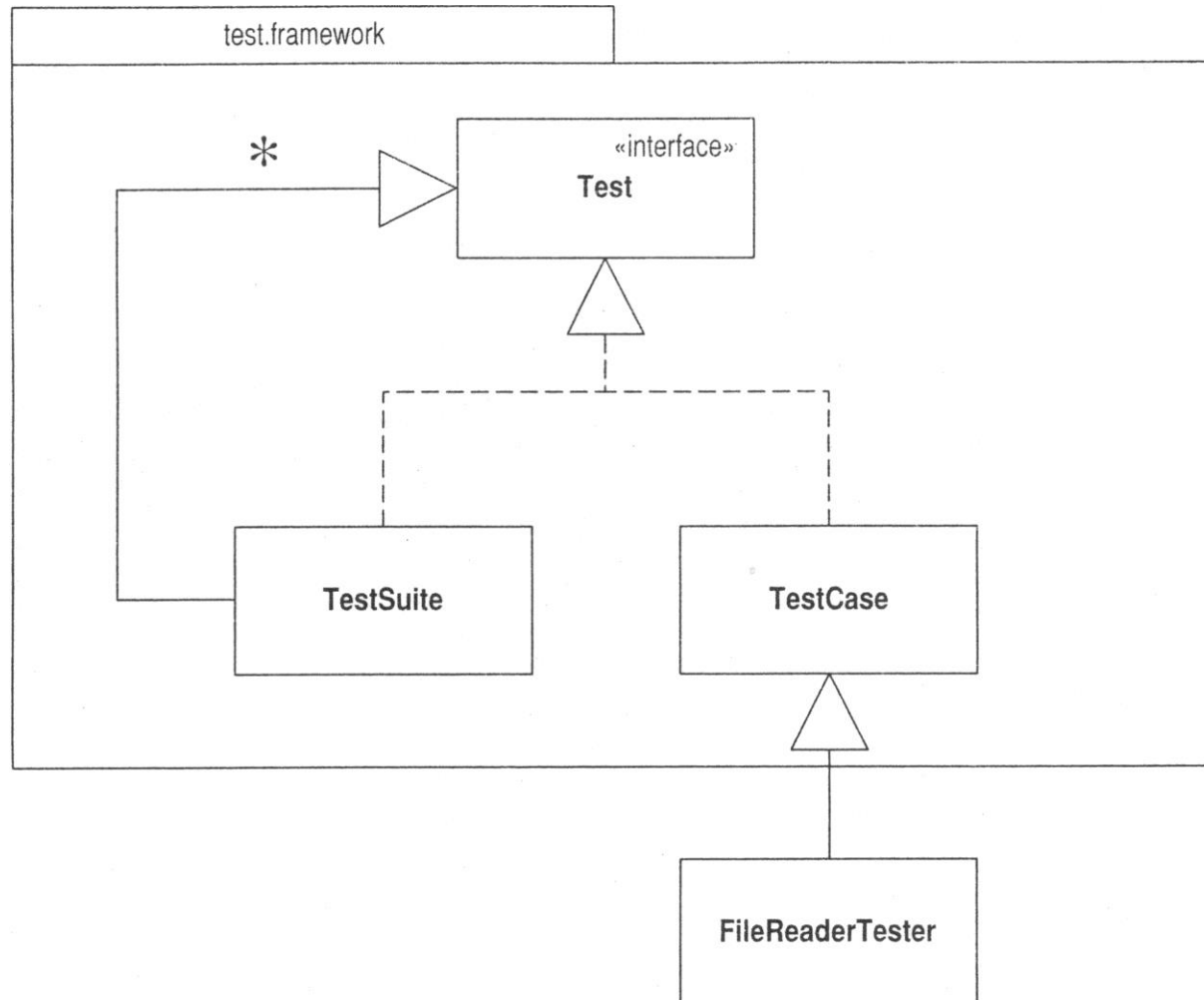
- **Detraction:** seem to be **writing code twice**
- Many programmers have never learned to write tests or even to think about tests
- Overhead of test framework

The Junit Testing Framework

- Used for writing **unit tests in Java**
- Helps **automate testing process**
- Provides some **basic constructs for running tests**



Structure of Junit Tests



Structure (cont'd)

- Any class that **contains tests must subclass the TestCase class**
 - Typically **one for each class being tested**
- **Junit framework** allows tests to be grouped into suites
 - TestSuites can **contain TestCases** or other TestSuites
 - Makes it easy to build a range of large test suites and run the tests automatically

Junit Example: I/O Class

- The **test must have a constructor**:

```
class FileReaderTester extends TestCase {  
    public FileReaderTester (String name) {  
        super(name);  
    }  
}
```

First step: Set up a test fixture

- A *test fixture* is the **set of objects that act as samples for testing**. In the case of **I/O testing**, a test file: data.txt

Bradman	99.94	52	80	10	6996	334	29
Pollock	60.97	23	41	4	2256	274	7
Headley	60.83	22	40	4	2256	270*	10
Sutcliffe	60.73	54	84	9	4555	194	16

Manipulating the test fixture

- **TestCase** provides:
 - protected void **setUp()** – creates objects
 - protected void **tearDown()** – removes them
- Important to **execute both methods for each test** so that the tests **are isolated from each other**; thus can run them in any order

setUp & tearDown

```
class FileReaderTester...
    protected void setUp() {
        try {
            _input = new FileReader("data.txt");
        } catch (FileNotFoundException e) {
            throw new RuntimeException ("unable to open test file");
        }
    }
    protected void tearDown(){
        try {
            _input.close();
        } catch (IOException e) {
            throw new RuntimeException ("error on closing test file");
        }
    }
}
```

Create the first test

```
public void testRead() throws IOException {  
    char ch;  
    for (int i = 0; i < 4; i++) {  
        ch = (char) _input.read();  
    }  
    assertEquals('d', ch);  
}
```

- **assertEquals** is the automatic Junit test

How to run the test

- **Create a test suite:**

```
class FileReaderTester ...
```

```
    public static Test suite(){
```

```
        TestSuite suite = new TestSuite();
```

```
        suite.addTest(new FileReaderTester("testRead"));
```

```
        return suite;
```

```
    }
```

- The test is **bound to the method testRead()**

How to run the test (cont'd)

- Use a separate **TestRunner** class
 - can use a **GUI version**, but **character interface can be called within the code**

```
class FileReaderTester ...  
    public static void main(String[] args) {  
        junit.textui.TestRunner.run(suite());  
    }
```

TestRunner success output

.

Time: 0.110

OK (1 tests)

- **Junit prints a period (“.”) for every test run**
- **Junit prints a single “OK” if no test fails**

TestRunner failure output

```
public void testRead() throws
    IOException
{
    char ch;
    for (int i = 0; i < 4; i++) {
        ch = (char) _input.read();
    }
    assertEquals('2', ch);
    // deliberate error
}
```

Result:

.F

Time: 0.220

FAILURES!!!

Test Results:

Run: 1 Failures: 1 Errors: 0

There was 1 failure:

1) FileReaderTester.testRead

“expected: “2” but was: “d”

Usefulness of Failures

- Can start by **making tests fail**, to prove:
 - the **test does actually run**
 - the **test is actually testing what it's supposed to**
- A **common testing error** is to be testing something other than what is supposed to be tested

Catching errors

- In addition to **catching failures** (assertions are false), **Junit's framework also catches errors** (unexpected exceptions)

```
public void testRead() throws IOException
{
    char ch;
    _input.close();
    for (int i = 0; i < 4; i++)
    {
        ch = (char) _input.read();
        // will throw exception
    }
    assertEquals('d', ch);
}
```

Result:

.E

Time: 0.110

!!!FAILURES!!!

Test Results:

Run: 1 Failures: 0 Errors: 1

There was 1 error:

1) FileReaderTester.testRead

java.io.IOException: Stream closed

Running multiple tests

- Write new test methods
 - public void **testReadAtEnd()**
- Put them in the suite to run them:
 - **suite.addTest(new FileReaderTester("testReadAtEnd"));**
- Junit has a **lazy-programmer shortcut**:
 - Naming convention: "testX()"
 - Replace main() method with:

```
public static void main(String[] args) {  
    junit.textui.TestRunner.run(new TestSuite(FileReaderTester.class));  
}
```

Can run a Master Test Suite

```
class MasterTester extends TestCase {  
    public static void main (String[] args) {  
        junit.textui.TestRunner.run (suite());  
    }  
    public static Test suite() {  
        TestSuite result = new TestSuite();  
        result.addTest(new TestSuite(FileReaderTester.class));  
        result.addTest(new TestSuite(FileWriterTester.class));  
        // and so on...  
        return result;  
    }  
}
```

User-defined comments in Junit

```
public void testReadBoundaries() throws IOException {  
    assertEquals("read first char", 'B', _input.read());  
    char ch;  
    for (int i = 1; i < 140; i++){  
        ch = _input.read();  
    }  
    assertEquals("read last char", '6', _input.read());  
    assertEquals("read at end", -1, _input.read());  
}
```


Testing philosophies

- **Testing should be risk-driven**
 - “test every public method” is not enough
- **A little testing goes a long way**
 - Better to **focus on complex code** and **areas that are at most risk of going wrong**
 - Helps to keep the **task of test-writing to a doable size**
- Focus on **boundary conditions** and **special conditions that make the test fail**
 - e.g. for an I/O class, an empty file

Conclusions

- **Cannot prove a program has no bugs** by testing
- **Tests will not find every bug**, but they will make it easier to find many bugs
- The **process of writing tests** sparks consideration of boundary and error conditions and helps with understanding interfaces